📘 Focus Hub: Documentación Técnica del Sistema

Versión del Sistema: 1.0 (Release Candidate)

Arquitectura: IoT Híbrido (Serverless + Embedded)

---

1. Visión General del Proyecto

Focus Hub es un ecosistema de productividad basado en el concepto de "Gemelo Digital". Combina una aplicación móvil moderna para la gestión de tareas con un dispositivo físico de escritorio (Hardware) que permite al usuario enfocarse en su trabajo mediante la técnica Pomodoro, minimizando las distracciones digitales.

El sistema garantiza una sincronización bidireccional casi instantánea entre el mundo físico y el digital utilizando la nube de AWS como orquestador central.

---

2. Arquitectura de Alto Nivel

El sistema utiliza una arquitectura desacoplada de tres capas:

1. **Capa de Usuario (App Móvil):** Interfaz para la creación y gestión administrativa de tareas.
2. **Capa de Hardware (Dispositivo IoT):** Interfaz física para la ejecución de tareas y control de tiempo.
3. **Capa de Nube (Backend Serverless):** Lógica de negocio, almacenamiento y mensajería en tiempo real.

Diagrama Conceptual

```
[ App Flutter] <--(HTTP/REST)--> [ AWS API Gateway/Lambda] <--(Actualización Sombra)--> [ AWS IoT Core] <--(MQTT)--> [ ESP32 FocusHub]
    |                                  |                                                                    |
    '---------> [ DynamoDB] <-----------'                                                                   |
                                                                                                            |
                          [ App Flutter] <--(Refresco Manual)-----------------------------------------------'
```

---

3. Detalle Técnico: Capa de Hardware (ESP32)

El dispositivo físico actúa como un terminal dedicado para la productividad.

3.1. Especificaciones de Hardware

- **Microcontrolador:** ESP32 (Dev Module).
- **Visualización:** Pantalla OLED 0.96" (Protocolo I2C).
- **Interfaz de Entrada:**
    - 3x Botones Push (Configuración Pull-Up interna).
    - 1x Potenciómetro 10k (Entrada Analógica para navegación).
- **Interfaz de Salida:**
    - 3x LEDs (Verde, Amarillo, Rojo) para feedback de estado.
    - 1x Buzzer Activo para notificaciones sonoras.

3.2. Lógica del Firmware (Versión 4.0)

El firmware está construido sobre una **Máquina de Estados Finitos** que gestiona la lógica interna y la comunicación asíncrona.

- **Gestión de Conectividad (Arranque):**
    - Conexión WiFi WPA2.
    - Autenticación mutua con AWS IoT Core (Certificados x.509).
    - **Suscripción Proactiva:** Al iniciar, se suscribe a $aws/.../get/accepted y solicita el estado completo para evitar iniciar con la lista vacía.
- **Estados Operativos:**
    - STATE_IDLE: Navegación de lista de tareas usando el potenciómetro.
    - STATE_FOCUS: Temporizador Pomodoro (25 min). LED Verde fijo.
    - STATE_BREAK: Temporizador Descanso (5 min). LED Amarillo fijo.
    - STATE_PAUSED: Cálculo matemático de tiempo restante (duracion - (ahora - inicio)). LED Amarillo parpadeante (sin delay bloqueante).
- **Gestión de Tareas (UI Optimista):**
    - Al completar una tarea manualmente (pulsación larga), el firmware la **elimina de la memoria local inmediatamente** para dar feedback visual instantáneo, mientras envía el mensaje MQTT focushub/complete en segundo plano.

---

4. Detalle Técnico: Capa de Nube (AWS Serverless)

La infraestructura es 100% *serverless*, lo que garantiza coste cero en reposo y escalabilidad automática.

4.1. Base de Datos (Amazon DynamoDB)

- **Tabla:** FocusHub_Tasks
- **Partition Key (PK):** userId (Agrupación por usuario).
- **Sort Key (SK):** taskId (Identificador único de tarea).
- **Atributos:** taskName, isComplete (Boolean), createdAt.

4.2. API Gateway & Seguridad

- **Protocolo:** REST API.
- **Recursos:** /tasks y /tasks/{taskId}.
- **Integración:** Proxy Lambda (Pasa la petición completa a la función).
- **CORS:** Habilitado para orígenes universales (*), permitiendo métodos GET, POST, PUT, OPTIONS.

4.3. Funciones Lambda (Orquestación)

El cerebro lógico del sistema se divide en micro-funciones:

1. **FocusHub_createTask (Sincronizador):**
    - Recibe la tarea vía HTTP.
    - Guarda en DynamoDB.
    - **Acción Crítica:** Consulta *todas* las tareas pendientes y actualiza la **Sombra de Dispositivo (Device Shadow)** en IoT Core. Esto empuja la tarea al ESP32 automáticamente.
2. **FocusHub_getTasks:**
    - Consulta DynamoDB y devuelve el JSON a la App.
3. **FocusHub_updateTask:**
    - Actualiza el estado (isComplete) en DynamoDB desde la App.
4. **FocusHub_markCompleteFromESP32 (Limpiador):**

    - Activada por regla IoT cuando el ESP32 termina una tarea.
    - Marca como completada en DynamoDB.
    - **Acción Crítica:** Vuelve a leer las pendientes y **sobrescribe la Sombra de IoT**. Esto evita el "efecto fantasma" (que la tarea reaparezca en el dispositivo tras ser borrada).

4.4. AWS IoT Core

- **Broker MQTT:** Gestiona la mensajería Pub/Sub.
- **Device Shadow:** Mantiene el estado "Deseado" (desired) de la lista de tareas, sirviendo como fuente de verdad para el ESP32.
- **Reglas:** Enruta mensajes del tópico focushub/complete hacia la Lambda de limpieza.

---

5. Detalle Técnico: Capa de Usuario (App Flutter)

Desarrollada con Flutter, compatible con Android.

5.1. Diseño y UX

- **Tema:** Dark Mode moderno ("Tech/Developer aesthetic") con colores BG: #0D1117 y Accent: #30D5C8.
- **Tipografía:** Google Fonts (Manrope).
- **Splash Screen Híbrida:** Utiliza flutter_native_splash para el arranque inmediato del sistema operativo, seguido de una animación personalizada en Flutter mientras carga datos asíncronos.

5.2. Gestión de Estado (Architecture)

- **Patrón:** MVVM simplificado usando Provider.
- **Actualizaciones Optimistas:**
    - Al marcar una tarea en la App, la UI cambia instantáneamente (antes de recibir respuesta de AWS).
    - Si la petición falla, se hace un "Rollback" automático al estado anterior y se notifica al usuario.

- **Servicio Desacoplado:** ApiService.dart maneja toda la lógica HTTP y serialización JSON, aislando la UI de la complejidad de la red.

---

6. Flujos de Datos Principales

A. Creación de Tarea (App -> Dispositivo)

1. **Usuario:** Crea tarea "Estudiar AWS" en la App.
2. **App:** Envía POST a API Gateway.
3. **Nube:** Lambda guarda en DB -> Actualiza Sombra IoT (desired: { tasks: [...] }).
4. **IoT:** Detecta cambio en Sombra -> Envía mensaje MQTT delta al ESP32.
5. **Dispositivo:** Recibe JSON -> Actualiza Pantalla OLED.
6. **Tiempo Total:** < 1 segundo.

B. Completar Tarea (Dispositivo -> Nube)

1. **Usuario:** Pulsación larga en botón físico del ESP32.
2. **Dispositivo (Optimista):** Borra tarea de pantalla inmediatamente -> Emite sonido de éxito.
3. **Dispositivo:** Envía mensaje MQTT focushub/complete.
4. **Nube:** Regla IoT activa Lambda -> Lambda marca true en DB -> Lambda limpia la tarea de la Sombra IoT.
5. **Resultado:** El sistema queda sincronizado y la tarea no vuelve a aparecer en el dispositivo.

---

7. Requerimientos de Despliegue

Credenciales de Hardware (secrets.h)

Para replicar el funcionamiento del hardware, se requieren:

- SSID y Password de WiFi (2.4GHz).
- Endpoint de AWS IoT.
- Certificado de CA de Amazon, Certificado del Dispositivo (.crt) y Llave Privada (.key).

Configuración de App (api_service.dart)

- URL base de invocación de AWS API Gateway (Stage dev).

Estado del Proyecto: Completado y Funcional.

Autor: Emiliano Espinoza Perales

Fecha: Noviembre 2025